DSA0108 – C++ PROGRAMMING.

LEVEL 1 QUESTIONS.

-ABISHANA SK

192511242 CSE

1. Check whether a number is a Harshad number

```
#include <iostream>
using namespace std;

bool isHarshad(int n) {
    int sum = 0, temp = n;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }
    return (n % sum == 0);
}

int main() {
    int num = 18;
    if (isHarshad(num)) cout << num << " is a Harshad number." << endl;
    else cout << num << " is not a Harshad number." << endl;
    return 0;
}
```

Output:

```
18 is a Harshad number.
```

2. Swap two numbers without using a temporary variable

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10;
    cout << "Before swap: a = " << a << ", b = " << b << endl;
    a = a + b;
    b = a - b;
    a = a - b;
    cout << "After swap: a = " << a << ", b = " << b << endl;
    return 0;
}
```

Output:

```
Before swap: a = 5, b = 10
After swap: a = 10, b = 5
```

3. Hollow square pattern

```
#include <iostream>
using namespace std;

void printHollowSquare(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1)
                cout << "* ";
            else
                cout << "  ";
        }
        cout << endl;
    }
}

int main() {
    printHollowSquare(5);
    return 0;
}
```

Output:

```
* * * * *
*       *
*       *
*       *
* * * * *
```

4. Number pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

5. Check whether a number is a palindrome

```
#include <iostream>
using namespace std;

bool isPalindrome(int n) {
    int rev = 0, temp = n;
    while (temp > 0) {
        rev = rev * 10 + (temp % 10);
        temp /= 10;
    }
    return (n == rev);
}

int main() {
    int num = 121;
    if (isPalindrome(num)) cout << num << " is a Palindrome." << endl;
    else cout << num << " is not a Palindrome." << endl;
    return 0;
}
```

Output:

```
121 is a Palindrome.
```

6. Find the sum of the cubes of the first n natural numbers

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += (i * i * i);
    }
    cout << "Sum of cubes of first " << n << " numbers: " << sum << endl;
    return 0;
}
```

Output:

```
Sum of cubes of first 5 numbers: 225
```

7. Convert binary to decimal

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;

int binaryToDecimal(string bin) {
    int dec = 0, base = 1;
    for (int i = bin.length() - 1; i >= 0; i--) {
        if (bin[i] == '1') dec += base;
        base *= 2;
    }
    return dec;
}

int main() {
    string bin = "1010";
    cout << "Decimal of binary " << bin << " is " << binaryToDecimal(bin) << endl;
    return 0;
}
```

Output:

```
Decimal of binary 1010 is 10
```

8. Palindrome pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++) cout << " ";
        for (int j = 1; j <= i; j++) cout << j;
        for (int j = i - 1; j >= 1; j--) cout << j;
        cout << endl;
    }
    return 0;
}
```

Output:

```
   1
  121
 12321
1234321
```

9. Hollow pyramid pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= (2 * n - 1); j++) {
            if (j == n - i + 1 || j == n + i - 1 || i == n)
                cout << "*";
            else
                cout << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
    *
   * *
  *   *
 *     *
*********
```

10. Find the sum of the squares of the first n natural numbers

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += (i * i);
    }
    cout << "Sum of squares of first " << n << " numbers: " << sum << endl;
    return 0;
}
```

Output:

```
Sum of squares of first 5 numbers: 55
```

11. Print all factors of a number

```
#include <iostream>
using namespace std;

int main() {
    int num = 24;
```

```
    cout << "Factors of " << num << ": ";
    for (int i = 1; i <= num; i++) {
        if (num % i == 0) cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Output:

```
Factors of 24: 1 2 3 4 6 8 12 24
```

12. Pyramid pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) cout << " ";
        for (int j = 0; j <= i; j++) cout << "* ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
    *
   * *
  * * *
 * * * *
* * * * *
```

13. Find the power of a number

```
#include <iostream>
using namespace std;

long long power(int base, int exp) {
    long long result = 1;
    for (int i = 0; i < exp; i++) result *= base;
    return result;
}

int main() {
    cout << "2 raised to power 5 is: " << power(2, 5) << endl;
    return 0;
}
```

Output:

```
2 raised to power 5 is: 32
```

14. Inverted pyramid pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
        for (int j = 0; j < n - i; j++) cout << " ";
        for (int j = 0; j < i; j++) cout << "* ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
* * * * *
 * * * *
  * * *
   * *
    *
```

15. Find the sum of positive and negative numbers from a list

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, -2, 3, -4, 5};
    int n = sizeof(arr)/sizeof(arr[0]);
    int posSum = 0, negSum = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] > 0) posSum += arr[i];
        else negSum += arr[i];
    }
    cout << "Positive Sum: " << posSum << ", Negative Sum: " << negSum << endl;
    return 0;
}
```

Output:

```
Positive Sum: 9, Negative Sum: -6
```

16. Generate a multiplication table

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= 10; i++) {
        cout << n << " x " << i << " = " << n * i << endl;
    }
    return 0;
}
```

Output:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
... (up to 5 x 10 = 50)
```

17. Check whether a number is an automorphic number

```
#include <iostream>
using namespace std;

bool isAutomorphic(int n) {
    int sq = n * n;
    int temp = n;
    while (temp > 0) {
        if (temp % 10 != sq % 10) return false;
        temp /= 10;
        sq /= 10;
    }
    return true;
}

int main() {
    int num = 25;
    if (isAutomorphic(num)) cout << num << " is an Automorphic number." << endl;
    else cout << num << " is not an Automorphic number." << endl;
    return 0;
}
```

Output:

```
25 is an Automorphic number.
```

19. Diamond pattern

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) cout << " ";
        for (int j = 0; j <= i; j++) cout << "* ";
        cout << endl;
    }
    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j < n - i - 1; j++) cout << " ";
        for (int j = 0; j <= i; j++) cout << "* ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
    *
   * *
  * * *
 * * * *
* * * * *
 * * * *
  * * *
   * *
    *
```

20. Convert hexadecimal to decimal and decimal to hexadecimal

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

int main() {
    int dec = 255;
    stringstream ss;
    ss << hex << uppercase << dec;
    cout << "Decimal 255 to Hex: " << ss.str() << endl;

    string hexStr = "FF";
    int decValue;
    stringstream ss2;
    ss2 << hex << hexStr;
    ss2 >> decValue;
    cout << "Hex FF to Decimal: " << decValue << endl;
    return 0;
}
```

Output:

```
Decimal 255 to Hex: FF
Hex FF to Decimal: 255
```

21. Inverted right triangle pattern

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) cout << "* ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
* * * * *
* * * *
* * *
* *
*
```

22. Check whether a number is a Strong number

```
#include <iostream>
using namespace std;

int fact(int n) {
    int f = 1;
    for (int i = 1; i <= n; i++) f *= i;
    return f;
}

int main() {
    int num = 145, sum = 0, temp = num;
    while (temp > 0) {
        sum += fact(temp % 10);
        temp /= 10;
    }
    if (sum == num) cout << num << " is a Strong number." << endl;
    else cout << num << " is not a Strong number." << endl;
    return 0;
}
```

Output:

```
145 is a Strong number.
```

23. Count the frequency of each digit in a number

```
#include <iostream>
using namespace std;

int main() {
    long long num = 1122333;
    int freq[10] = {0};
    long long temp = num;
    while (temp > 0) {
        freq[temp % 10]++;
        temp /= 10;
    }
    cout << "Digit Frequencies inside " << num << ":" << endl;
    for(int i=0; i<10; i++) {
        if(freq[i] > 0) cout << i << " occurs " << freq[i] << " times" << endl;
    }
    return 0;
}
```

Output:

```
Digit Frequencies inside 1122333:
1 occurs 2 times
2 occurs 2 times
3 occurs 3 times
```

24. Count the number of divisors of a number

```
#include <iostream>
using namespace std;

int main() {
    int num = 12, count = 0;
    for (int i = 1; i <= num; i++) {
        if (num % i == 0) count++;
    }
    cout << "Number of divisors of " << num << ": " << count << endl;
    return 0;
}
```

Output:

```
Number of divisors of 12: 6
```

25. Calculate compound interest

```
#include <iostream>
#include <cmath>
using namespace std;
```

```
int main() {
    double principal = 10000, rate = 5, time = 2;
    double amount = principal * pow((1 + rate / 100), time);
    double ci = amount - principal;
    cout << "Compound Interest: " << ci << endl;
    return 0;
}
```

Output:

```
Compound Interest: 1025
```

26. Reverse the digits of a number

```
#include <iostream>
using namespace std;

int main() {
    int num = 1234, rev = 0;
    while (num > 0) {
        rev = rev * 10 + (num % 10);
        num /= 10;
    }
    cout << "Reversed Number: " << rev << endl;
    return 0;
}
```

Output:

```
Reversed Number: 4321
```

27. Check whether a number is a Perfect number

```
#include <iostream>
using namespace std;

int main() {
    int num = 6, sum = 0;
    for (int i = 1; i < num; i++) {
        if (num % i == 0) sum += i;
    }
    if (sum == num) cout << num << " is a Perfect number." << endl;
    else cout << num << " is not a Perfect number." << endl;
    return 0;
}
```

Output:

```
6 is a Perfect number.
```

28. Generate the Fibonacci series

```
#include <iostream>
using namespace std;

int main() {
    int n = 10, a = 0, b = 1, next;
    cout << "Fibonacci series: ";
    for (int i = 0; i < n; i++) {
        cout << a << " ";
        next = a + b;
        a = b;
        b = next;
    }
    cout << endl;
    return 0;
}
```

Output:

```
Fibonacci series: 0 1 1 2 3 5 8 13 21 34
```

29. Find the LCM of two numbers

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int main() {
    int a = 12, b = 18;
    int lcm = (a * b) / gcd(a, b);
    cout << "LCM of " << a << " and " << b << " is " << lcm << endl;
    return 0;
}
```

Output:

```
LCM of 12 and 18 is 36
```

30. Check whether a number is a Magic number

```
#include <iostream>
using namespace std;

bool isMagic(int n) {
    int sum = 0;
    while (n > 0 || sum > 9) {
```

```
        if (n == 0) {
            n = sum;
            sum = 0;
        }
        sum += n % 10;
        n /= 10;
    }
    return (sum == 1);
}

int main() {
    int num = 19;
    if (isMagic(num)) cout << num << " is a Magic number." << endl;
    else cout << num << " is not a Magic number." << endl;
    return 0;
}
```

Output:

```
19 is a Magic number.
```

31. Convert decimal to binary

```
#include <iostream>
#include <string>
using namespace std;

string decToBinary(int n) {
    string bin = "";
    while (n > 0) {
        bin = to_string(n % 2) + bin;
        n /= 2;
    }
    return bin == "" ? "0" : bin;
}

int main() {
    int num = 10;
    cout << "Binary of " << num << " is " << decToBinary(num) << endl;
    return 0;
}
```

Output:

```
Binary of 10 is 1010
```

32. Find the repeated sum of digits (digital root)

```
#include <iostream>
using namespace std;
```

```
int digitalRoot(int n) {
    if (n == 0) return 0;
    return 1 + (n - 1) % 9;
}

int main() {
    int num = 9875;
    cout << "Digital root of " << num << " is " << digitalRoot(num) << endl;
    return 0;
}
```

Output:

```
Digital root of 9875 is 2
```

33. Calculate the average of given numbers

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr)/sizeof(arr[0]);
    double sum = 0;
    for (int i = 0; i < n; i++) sum += arr[i];
    cout << "Average: " << sum / n << endl;
    return 0;
}
```

Output:

```
Average: 30
```